

35.2. La fonction `sprite16()` — un personnage de 16×16

`#include <sprite16>` : `sprite16(numero, x, y, PERSON)` place un personnage de 16×16 — quatre lutins en carré — d'un appel. `BONHOMME` et `ROBOT`, chacun de son fichier, descendent ensemble.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `sprite16(numero, x, y, perso)` place un personnage de 16×16 au pixel près : les lutins `numero` à `numero + 3`, en carré, par-dessus le fond. Un seul appel, au lieu des quatre `poser()` du 35.1 — et il peut bouger.

La ligne à écrire : `#include <sprite16>`.

Ses arguments : le numéro du **premier** des quatre lutins, `x` et `y` (le coin haut-gauche, en pixels), et le **nom** du personnage. Un cinquième, `MIROIR_X`, le retourne.

Deux personnages, deux appels, chacun ses lutins : `sprite16(0, 40, y, BONHOMME)` prend les lutins 0 à 3 ; `sprite16(4, 104, y, ROBOT)` les lutins 4 à 7. Le second commence à 4 : un personnage de 16×16 occupe **quatre** lutins, il ne faut pas qu'ils se chevauchent.

Les personnages viennent de leurs fichiers, ceux du 35.1, sans changement : la liste `personnages.cpp` les inclut, et `principal.cpp` la verse d'une ligne.

Déroulé : `y` part de 0 et grandit d'un pixel à chaque image jusqu'à 120 ; à chaque tour, les deux `sprite16` les reposent à la hauteur `y` : ils descendent ensemble, et s'arrêtent en bas.

Ce qu'elle coûte : environ **270 octets** avec ses dessins.

Pour un personnage de 32×32 , c'est `sprite32()`, avec seize lutins.

Essaie : fais descendre ROBOT moins vite : `sprite16(4, 104, y / 2, ROBOT)`.

On retrouve `sprite16` dans 64 leçons

[35.6](#), [35.7](#), [35.8](#), [35.9](#), [35.10](#), [35.12](#), [35.16](#), [35.17](#), [35.18](#), [35.19](#), [36](#), [47](#), [2.01](#), [2.02](#), [2.03](#), [2.04](#), [2.05](#), [2.06](#), [2.07](#), [2.08](#), [2.09](#), [2.10](#), [2.11](#), [2.12](#), [2.13](#), [2.14](#), [2.15](#), [2.16](#), [2.17](#), [2.18](#), [2.19](#), [2.20](#), [2.21](#), [2.22](#), [2.23](#), [2.24](#), [2.25](#), [2.26](#), [2.27](#), [2.28](#), [2.29](#), [2.30](#), [2.31](#), [2.32](#), [2.33](#), [2.34](#), [2.37](#), [3.01](#), [3.02](#), [3.03](#), [3.04](#), [3.05](#), [3.06](#), [3.07](#), [3.08](#), [3.09](#), [3.10](#), [3.11](#), [3.12](#), [3.13](#), [3.14](#), [3.15](#), [3.16](#), [3.17](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **520 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      50 o ← la console, à l'allumage – la mise en route de la console
main()         223 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
RangerLutins   11 o ← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles   31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
Tuiles         128 o ← CopierTuiles – 0 lettre sur 43, et 8 tuiles dessinées
DonneesTransfert 10 o ← InstallerTransfert – la petite routine qui copie les lutins
```

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

```
sprite()  × 8   200 o sur place, dans main()
image()   × 1    6 o sur place, dans main() + AttendreImage (28 o)
sprite16() × 2    0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <sprite16> : un personnage de 16 × 16 ----
// BONHOMME et ROBOT (chacun dans son fichier) descendent d'un pixel par image.

#include <sprite16>          // sprite16() : quatre lutins en carré
#include "personnages.cpp"   // la liste : BONHOMME et ROBOT
```

2. UNE VARIABLE — Y

```
uint8_t y = 0; // la hauteur des deux personnages, en pixels
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

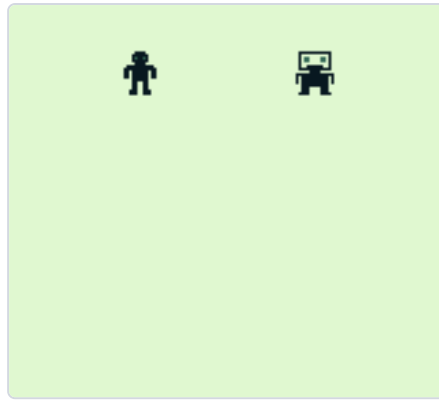
```
int main() {
    while (true) {
        image();
        if (y < 120) y = y + 1;           // ils descendent, et s'arrêtent en 120
        sprite16(0, 40, y, BONHOMME);    // BONHOMME : les lutins 0, 1, 2, 3
        sprite16(4, 104, y, ROBOT);       // ROBOT : les lutins 4, 5, 6, 7
    }
}
```

3. Ce qui se passe, étape par étape

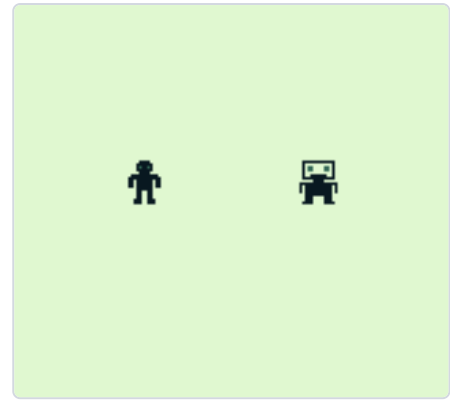
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



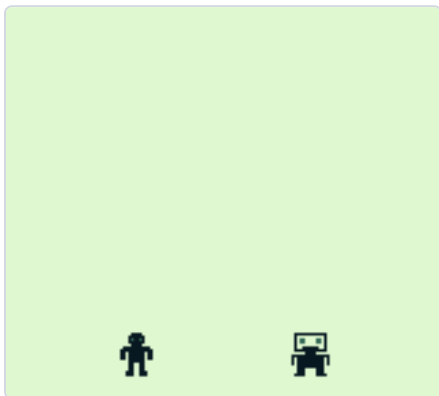
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le bonhomme et le robot descendent ensemble, côte à côte, et s'arrêtent en bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ BONHOMME est arrivé en $y = 120$ (40, 120)
- ✓ ses quatre lutins font un carré de 16
- ✓ ROBOT, lutins 4 à 7, en (104, 120)
- ✓ chacun avec ses tuiles

Ensuite : **35.3. La fonction `cacher16()` — ôter un personnage de 16×16** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++